PAI-4. DEVSECOPS - ASEGURAMIENTO DE LA CADENA DE SUMINISTRO EN EL DESARROLLO DE APLICACIONES WEB

CONFIDENCIAL

Cliente	Entidad de desarrollo de aplicaciones web
Fecha	26/04/2026
Autores	Emilio Vázquez Cruz Javier Ignacio Milá de la Roca Dos Santos José Coronil Álvarez

Índice

Índice 2
Resumen ejecutivo 2
Especificación técnica 3
Manual de despliegue 4
Inicialización de DefectDojo 4
Inicialización del proyecto 5
Resolución de incidencias 6
Grado de completitud 8

Resumen ejecutivo

Se han integrado pruebas SCA con OWASP Dependency Check, SAST con Bearer y DAST con Wapiti en el flujo de desarrollo e integración contínua para establecer prácticas DevSecOps. Se ha conseguido usando workflows de GitHub Actions.

Adicionalmente, se han conectado los resultados con el gestor de vulnerabilidades DefectDojo, que se ha usado para gestionar la solventación de una incidencia encontrada con el análisis SCA.

CONFIDENCIAL

Especificación técnica

Se ha creado una aplicación de ejemplo simple con de ExpressJS con una versión desactualizada y vulnerable de la biblioteca `lodash`. Está alojada en un repositorio de GitHub para poder usar el pipeline de GitHub Actions.

Se ejecutan las siguientes tres herramientas:

- OWASP Dependency Check, un SCA que busca vulnerabilidades en la plataforma del NIST NVD.
- Bearer, un SAST que analiza el código para detectar las vulnerabilidades top 10 de OWASP, las top 25 del CWE, etc.
- Wapiti, un DAST black-box que prueba endpoints con un fuzzer para detectar inyección SQL, XSS, ejecución de comandos, inyección CRLF, etc.

Los resultados de estas herramientas se transmitirán a DefectDojo, un gestor de vulnerabilidades, alojado localmente con Docker y expuesto a internet por un tunel con `ngrok` para que sea accesible desde el ejecutor de las pipelines. Para esto se usará su API, sencillamente ejecutando `curl` desde el workflow.

```
run: |
  curl -X POST "$DD_URL/api/v2/import-scan/" \
    -H "Authorization: Token $DD_API_KEY" \
    -F "scan_type=Generic Findings Import" \
    -F "file=@bearer-report.json" \
    -F "engagement=$DD_ENGAGEMENT_ID"
```

La URL de su instancia de DefectDojo y su clave API se almacenan con secretos de GitHub.

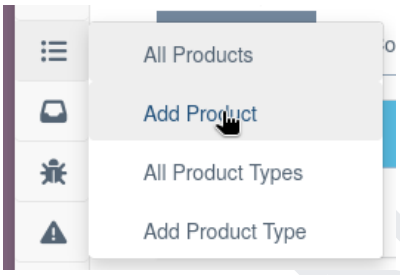
```
env:
  DD_URL: ${{ secrets.DD_URL }}
  DD_API_KEY: ${{ secrets.DD_API_KEY }}
  DD_ENGAGEMENT_ID: ${{ vars.DD_ENGAGEMENT_ID }}
```

CONFIDENCIAL

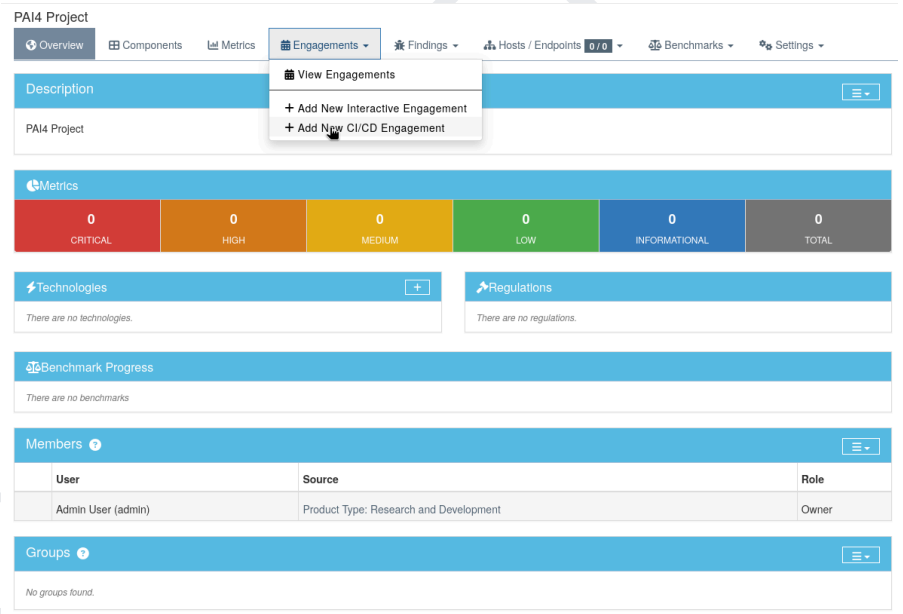
Manual de despliegue

Inicialización de DefectDojo

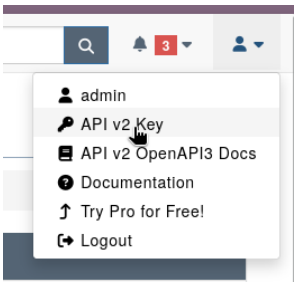
1. Clone e inicie DejectDojo, asegurándose de anotar la contraseña del administrador que aparece durante la inicialización bajo `Admin password: ...`
   ```
   git clone https://github.com/DefectDojo/django-DefectDojo
   cd django-DefectDojo
   docker compose up
   ```
2. Inicie sesión en `http://localhost:8080` con el nombre de usuario `admin` y la contraseña que anotó.
3. Cree un Product llamado `PAI4` en la sección de añadir producto.

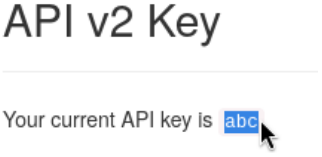


4. Añada un Engagement de tipo CI/CD.



5. En la URL, copie el ID del Engagement. Por ejemplo, si su URL es `http://localhost:8080/engagement/1`, la ID es `1`.
6. En la vista de usuario, copie su clave API v2:

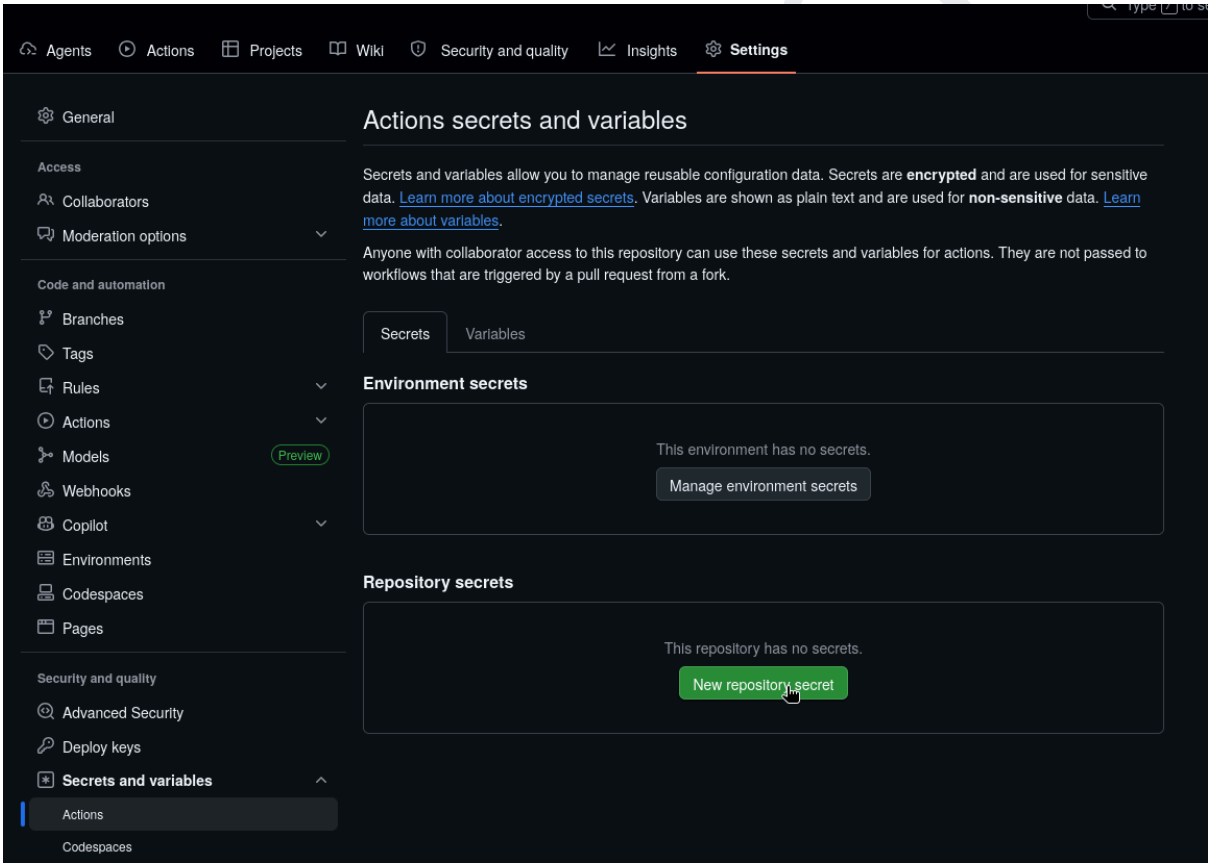




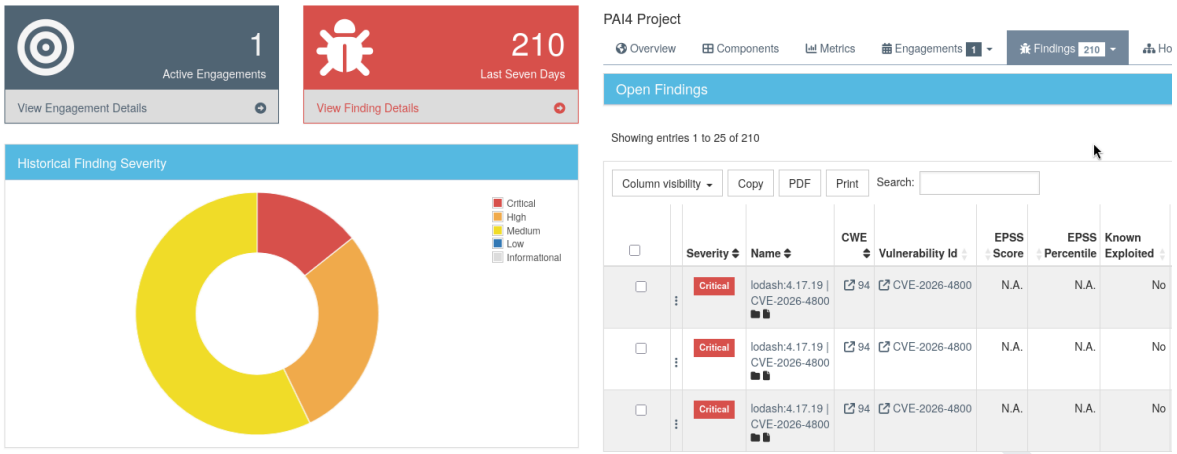
7. Exponga su instancia de DefectDojo a internet. Recomendamos usar la herramienta ngrok. Para hacerlo, ejecute `ngrok http 8080`, diríjase a la dirección que le indique como "Web interface" (p. ej. `http://127.0.0.1:4040`), y copie la URL de su tunel (p. ej. `https://a-b-c.ngrok-free.dev`).

Inicialización del proyecto

1. En GitHub, cree un fork de https://github.com/miniluz/pai4, y clónelo a su ordenador: `git clone https://github.com/usted/pai4`. Alternativamente, cree un repositorio git con el código que se le proporciona y súbalo a GitHub manualmente.
2. En Settings > Secrets and Variables > Actions, defina los siguientes secretos para los workflows, bajo Secrets > Repository secrets:
   `DD_URL=https://a-b-c.ngrok-free.dev` (su URL de ngrok)
   `DD_API_KEY=abc` (su clave API v2)
   Y la siguiente variable, bajo Variables > Repository variables:
   `DD_ENGAGEMENT_ID=1` (la ID de su Engagement)

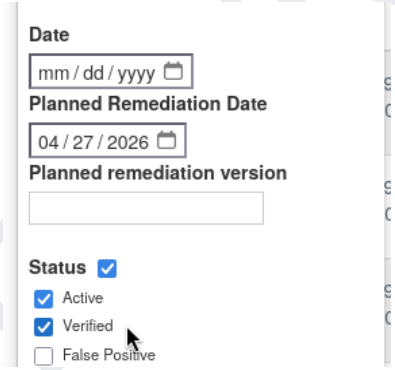


3. Haga un commit cualquiera y pushee al remoto para ejecutar los workflows (p. ej. `touch foo && git add -A && git commit -m "Foo" && git push`). Esto automáticamente ejecuta los workflows.
4. Espere unos minutos a que se ejecute la acción. Puede revisar su estado en la pestaña "Actions".
5. Al ver su proyecto en DefectDojo en `http://localhost:8080` , podrá ver las vulnerabilidades que han sido detectadas.

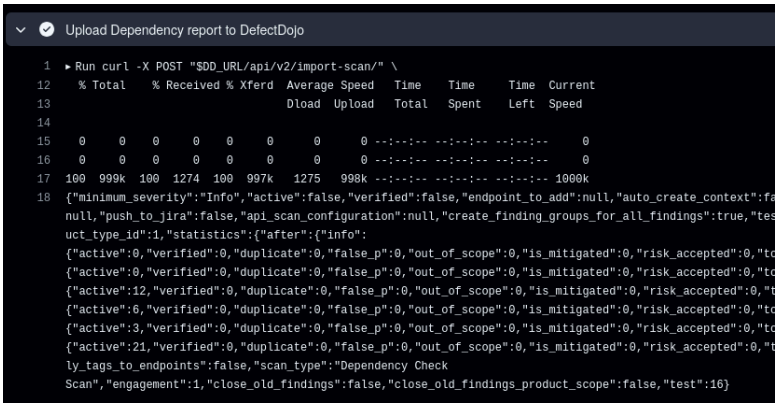


Resolución de incidencias

Como se puede ver, el proyecto tiene muchas incidencias relacionadas a Lodash. Estos problemas son reales, así que los marcaremos como activos y verificados y pondremos una fecha de remediación, como se muestra. No tocaremos la prioridad, ya que Dependency Check las ha priorizado automáticamente de acuerdo a la severidad.



En la versión entregada, este problema ya ha sido arreglado, por lo que si se quiere recrear se puede ir a `package.json` y bajo `dependencies` especificar `"lodash": "^4.17.19"`, ejecutar `npm install`, hacer commit y pushear. También se pueden ver los resultados en una [ejecución](#) previa al arreglo:

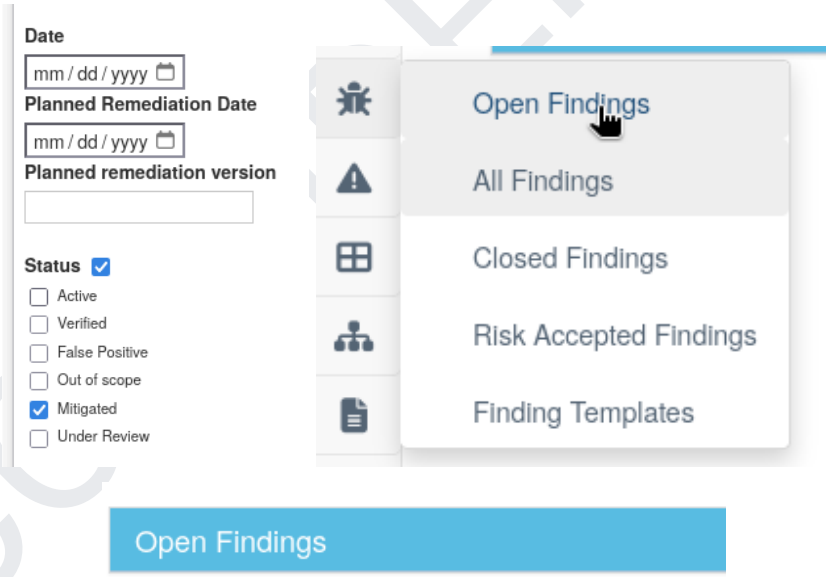


CONFIDENCIAL

Se actualiza la versión de `lodash`: `npm i lodash`. Se hace commit y pushea: `git add -A && git commit -m "Actualizar Lodash" && git push`. Veremos que ya no se han encontrado vulnerabilidades ya que no se añaden a DefectDojo, pero se puede revisar manualmente en los detalles de la ejecución:

```
Upload Dependency report to DefectDojo                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                1s
1  ▸ Run curl -X POST "$DD_URL/api/v2/import-scan/" \
12   % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
13                                  Dload  Upload   Total   Spent    Left  Speed
14
15   0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
16 100  815k  100  1270  100  814k   1082   694k  0:00:01  0:00:01 --:--:--  695k
17 100  815k  100  1270  100  814k   1082   694k  0:00:01  0:00:01 --:--:--  695k
18 {"minimum_severity":"Info","active":false,"verified":false,"endpoint_to_add":null,"auto_create_context":false,"deduplication_on_engagement":false,"lead":
   null,"push_to_jira":false,"api_scan_configuration":null,"create_finding_groups_for_all_findings":true,"test_id":19,"engagement_id":1,"product_id":1,"prod
   uct_type_id":1,"statistics":{"after":{"info":
   {"active":0,"verified":0,"duplicate":0,"false_p":0,"out_of_scope":0,"is_mitigated":0,"risk_accepted":0,"total":0},"low":
   {"active":0,"verified":0,"duplicate":0,"false_p":0,"out_of_scope":0,"is_mitigated":0,"risk_accepted":0,"total":0},"medium":
   {"active":0,"verified":0,"duplicate":0,"false_p":0,"out_of_scope":0,"is_mitigated":0,"risk_accepted":0,"total":0},"high":
   {"active":0,"verified":0,"duplicate":0,"false_p":0,"out_of_scope":0,"is_mitigated":0,"risk_accepted":0,"total":0},"critical":
   {"active":0,"verified":0,"duplicate":0,"false_p":0,"out_of_scope":0,"is_mitigated":0,"risk_accepted":0,"total":0},"total":
   {"active":0,"verified":0,"duplicate":0,"false_p":0,"out_of_scope":0,"is_mitigated":0,"risk_accepted":0,"total":0}}},"apply_tags_to_findings":false,"apply
   _tags_to_endpoints":false,"scan_type":"Dependency Check
   Scan","engagement":1,"close_old_findings":false,"close_old_findings_product_scope":false,"test":19}
```

Podemos ahora marcarlos como mitigados, y al ir a la página de Open Findings podemos verificar que no queda ninguno abierto.



Grado de completitud

1. Definir una Pipeline CI/CD en un proyecto de prueba en un repositorio de código (SCM)

`.github/workflows/pipeline.yml`
https://github.com/miniluz/pai4/blob/main/.github/workflows/pipeline.yml

2. Seleccionar al menos tres herramientas de Testeo de Seguridad (SCA, SAST, IAST, DAST, Secuity IaC) para incluirlas en el Pipeline para dotarlos de capacidades DevSecOps

`.github/workflows/pipeline.yml`
https://github.com/miniluz/pai4/blob/main/.github/workflows/pipeline.yml

Incluyen:

- OWASP Dependency Check (SCA)
- Bearer (SAST)
- Wapiti (DAST)

3. Integrar las herramientas de Testeo en el ciclo de vida del desarrollo para hacer Pipeline DevSecOps en un ciclo de integración continua

`.github/workflows/pipeline.yml`
https://github.com/miniluz/pai4/blob/main/.github/workflows/pipeline.yml

Se ejecuta a empujar a main, la única rama usada en la metodología de desarrollo de este proyecto. Contínuamente se integran cambios a main, y se prueban en la pipeline.

4. Desarrollar los tests correspondientes que permitan detectar vulnerabilidades en cada etapa del Pipeline DevSecOp

`.github/workflows/pipeline.yml`
https://github.com/miniluz/pai4/blob/main/.github/workflows/pipeline.yml

5. Seleccionar una herramienta de gestión de vulnerabilidades para integrar, priorizar y clasificar las vulnerabilidades de Seguridad que se vayan encontrando por las herramientas de testeo en el Pipeline DevSecOps. Como recomendación podría usarse DefectDojo o cualquier otra herramienta que se considere oportuna

`.github/workflows/pipeline.yml`
https://github.com/miniluz/pai4/blob/main/.github/workflows/pipeline.yml

Se puede ver la integración con DefectDojo con CURL, como se describe en el reporte técnico, además de en las capturas incluídas en el proyecto.

CONFIDENCIAL